Guía1. Definición Proyecto APT Asignatura Capstone

A. PARTE I

1. Antecedentes Personales

A continuación, se presenta una tabla en la que debes completar la información solicitada.

Nombre estudiante	Benjamín Herrera y Julián Tapia
Carrera	Ingeniería en Informática
Sede	San Joaquin

2. Descripción Proyecto APT

En la descripción debes señalar brevemente el nombre de tu proyecto APT y las competencias del perfil de egreso que vas a poner en práctica. Si en tu carrera están definidas las áreas de desempeño, también menciona a qué áreas de desempeño está vinculado el proyecto.

Nombre del proyecto	Coherev – Intranet corporativa
Área (s) de desempeño(s)	Desarrollo de software (aplicaciones web), modelado y gestión de datos, gestión de proyectos
Competencias	Construir programas y rutinas de variada complejidad, desarrollar una solución de software, construir modelos de datos, programar consultas o rutinas para manipular información de una base de datos, Construir el modelo arquitectónico de una solución sistémica, resolver las vulnerabilidades sistémicas y gestionar proyectos informáticos.

3. Fundamentación Proyecto APT

A continuación, se presentan distintos campos que debes completar con la información solicitada. Esta sección busca que describas en detalle tu proyecto y justifiques su relevancia y pertinencia.

Relevancia del proyecto APT	¿Por qué escogiste este tema? ¿Por qué es relevante este tema para el campo laboral de tu carrera? Lo escogimos porque no fue una idea inventada, sino un problema que vimos de cerca. Benjamín trabajó como junior software developer en Nomed, un startup con integrantes en Chile, México, Perú y Rusia, y ahí se hizo evidente que no existía ninguna forma de ordenar la información interna: la comunicación se repartía entre correo, mensajería y carpetas compartidas, no había control de acceso y nadie sabía con certeza quién tenía pendiente qué. El tema es relevante porque ese problema no es exclusivo de un startup. Cualquier organización que trabaje con equipo distribuido enfrenta lo mismo, y desde el lado técnico es exactamente el tipo de sistema que se pide en el campo: autenticación, control de permisos, base de datos relacional, manejo de archivos y comunicación en tiempo real. Es desarrollo full stack completo, no una pantalla con datos simulados. ¿Dónde se ubica la situación que vas a abordar? ¿Cuáles son las características principales de ese lugar? El cliente es Nomed, un startup con equipo repartido entre Chile, México, Perú y Rusia. La característica que define su forma de trabajar es precisamente esa distribución: la distancia física hace que la comunicación por vías informales deje de funcionar y que la información se pierda. Para efectos del proyecto académico, el alcance se ajusta a una intranet funcional para una organización mediana con esa misma característica: equipo distribuido, departamentos definidos y usuarios con distintos permisos. ¿A quiénes afecta o impacta la situación que abordaste? • A los colaboradores, que son los más afectados: no tienen una fuente única de qué se les pidió, dependen de mensajes sueltos para enterarse de sus tareas y pierden tiempo buscando información que sí existe, pero está en otro lugar. • A los responsables de área, que necesitan conocer el estado del trabajo sin tener que preguntar uno por uno. • A la organización en general, de forma indirecta: cuando la información está dispersa se duplica el trabajo y se pierde la trazabilidad de qué se decidió y cuándo. ¿Cuál sería el aporte de valor (real o simulado) de tu Proyecto APT? El aporte es simulado en cuanto a la instalación —el sistema no se despliega en la empresa cliente, porque el proyecto es académico y corre en entorno local— pero real en cuanto a los problemas que resuelve: • Una fuente única de información: mensajería interna, publicaciones y documentos en un mismo lugar.

	• Control de acceso efectivo: cuatro roles canónicos con permisos diferenciados, de modo que un colaborador no vea conversaciones ni documentos que no le corresponden. • Trazabilidad de lectura: el sistema registra qué mensajes se han leído, en base de datos, por lo que funciona entre dispositivos. • Documentación centralizada: carpetas por departamento y control de visibilidad por archivo (público, departamental o privado).
Descripción del Proyecto APT	¿Qué se espera lograr con el proyecto y en qué consistiría? Se espera desarrollar una intranet corporativa que centralice la comunicación interna, la gestión documental y el control de acceso, para una organización con equipo distribuido. El proyecto consiste en construir una aplicación web con dos aplicaciones independientes en un mismo repositorio: • El servidor (backend), construido en Laravel, que expone una API REST y se ocupa de la lógica de negocio, la persistencia, la autenticación y la emisión de eventos en tiempo real. La base de datos es relacional, con el esquema definido desde el inicio mediante migraciones versionadas. • La interfaz (frontend), construida en Next.js, que consume esa API y se comunica además por WebSocket para la mensajería instantánea. El alcance concreto incluye: autenticación con tokens de acceso y cierre de sesión; cuatro roles canónicos (Administrador, Líder, Colaborador, Nuevo Ingreso) con permisos diferenciados; mensajería interna en tiempo real entre conversaciones, con adjuntos y contador de mensajes no leídos persistido en base de datos; publicaciones internas con comentarios y archivos adjuntos; gestión documental con carpetas jerárquicas por departamento y control de visibilidad por archivo; y administración de usuarios, roles y departamentos con una papelera de recuperacion para lo que se elimina. La manera de abordarlo es incremental: primero el modelo de datos y la autenticación, después la autorización y los roles, luego la funcionalidad de comunicación y documentos, y finalmente la validación y el pulido. Cada etapa se entrega funcionando y verificada, no como una parte pendiente.
Pertinencia del proyecto con el perfil de egreso	¿De qué manera se relaciona el Proyecto APT con el perfil de egreso y con las competencias seleccionadas? El proyecto se relaciona con el perfil de egreso porque el Ingeniero Informático debe ser capaz de analizar, diseñar, desarrollar e integrar sistemas computacionales, y eso es exactamente lo que hace una intranet con control de acceso: un sistema completo, no un ejercicio puntual. **Competencias** • **Construir programas y rutinas de variada complejidad:** Desarrollo de los controladores, la lógica de negocio, el cliente HTTP del frontend y la conexión en tiempo real. • **Desarrollar una solución de software:** Se pone en práctica el ciclo completo: definición de requerimientos, modelo de datos, construcción, integración y mantenimiento.

	• **Construir modelos de datos:** Diseño del esquema relacional (usuarios, roles, departamentos, conversaciones, mensajes, publicaciones, comentarios, documentos) y sus relaciones. • **Programar consultas o rutinas para manipular información:** Consultas con relaciones y filtros, y el contador de mensajes no leídos, que requiere agregación sobre la tabla de lecturas. • **Construir el modelo arquitectónico de una solución sistémica:** Separación en capas (comunicación, dominio, persistencia, autorización, tiempo real) y la decisión de separar el cliente y el servidor en dos aplicaciones que se comunican por API y WebSocket. • **Resolver las vulnerabilidades sistémicas:** Control de acceso por rol, autorización a nivel de rutas y de modelo, validación de la extensión real de los archivos, límite de intentos de inicio de sesión y almacenamiento de contraseñas y tokens con hash. • **Gestionar proyectos informáticos:** Planificación por iteraciones, control de versiones, reparto de responsabilidades y seguimiento del avance. La competencia que más se pone a prueba es resolver las vulnerabilidades sistémicas, porque en una intranet lo que se expone no es un dato cualquiera: son documentos de la empresa y conversaciones internas. Un error de autorización no es un detalle, es una fuga.
Relación con los intereses profesionales	¿Cuáles son tus intereses profesionales y qué aspectos de ellos se reflejan en el proyecto? **Benjamín**. Mi interés está en el desarrollo backend y en el camino hacia la arquitectura de software, y el proyecto se apoya justamente en eso: la decisión de separar las responsabilidades, centralizar la lógica de permisos y modelar bien la base de datos es trabajo de arquitectura. Además, la parte de seguridad es la que más me interesa del proyecto, porque en un trabajo real es lo que más se nota. Me aporta mucho, porque la experiencia de la práctica en Nomed me dejó claro que el valor no está solo en que la aplicación funcione, sino en que nadie acceda a lo que no le corresponde. **Julian**. Mi interés está en el desarrollo de aplicaciones, especialmente en el frontend y el UI/UX, y el proyecto pone eso en el centro: las siete vistas del dashboard, el sistema de navegación que se adapta al rol y el recorrido de bienvenida para el usuario nuevo. Pero me está haciendo crecer en otra dirección: entender el modelo de datos y las reglas de autorización, porque una interfaz puede verse perfecta y ser inútil si el servidor no la sostiene. Quiero que este proyecto me deje capaz de construir un proyecto completo, no solo la parte visual. ¿De qué manera va a contribuir a nuestro desarrollo profesional? Nos va a dar un caso real de un sistema que se despliega de punta a punta y que podremos mostrar: código, documentación de arquitectura, base de datos y pruebas. Y nos va a obligar a trabajar con las herramientas del oficio —control de versiones, revisión de código, documentación continua— que es lo que después se espera en cualquier equipo de desarrollo.
Factibilidad de desarrollo del Proyecto APT	¿Por qué es posible desarrollar tu Proyecto APT? 1. Duración del semestre. El periodo académico completo (18 semanas) se ordena en las tres fases de la asignatura, y el alcance se ajustó a ese tiempo:

	dos iteraciones de seguridad y roles sobre el núcleo ya construido, más tres módulos funcionales nuevos, documentación y validación. 2. Horas asignadas a la asignatura. Es lo que más condiciona el alcance, y por eso partimos con una prioridad explícita: primero que el sistema exista y esté protegido, después que crezca en funcionalidad. Elegimos dejar fuera elementos que requieren más tiempo del disponible, en vez de entregar siete módulos a medias. 3. Materiales requeridos. Todo el stack es gratuito y corre en la máquina: PHP, Composer, Node.js, un gestor de paquetes, la base de datos MariaDB y el servidor de WebSocket. No hay licencias, ni servidores de pago, ni servicios cloud que cuesten. Las herramientas de apoyo son el editor de código, el gestor de tareas y el repositorio remoto. 4. Factores externos que facilitan el desarrollo. El equipo ya trabaja con Git y GitHub de forma ordenada, con ramas por funcionalidad y revisión cruzada, lo que permite trabajar en paralelo sin pisarse. Los dos conocemos bien el stack que elegimos, no estamos aprendiendo dos tecnologías a la vez. La IA generativa sirve para resolver errores y comparar alternativas de implementación sin que dejemos de entender el código. 5. Factores externos que dificultan el desarrollo y cómo los resolveríamos. El principal es el tiempo: una parte importante de lo que nos faltaba era de seguridad y autorización, y es la clase de trabajo que más se estira porque hay que revisar todos los caminos de acceso, no solo el principal. La estrategia fue acotar el alcance de las tarjetas, aceptando un nivel de detalle razonable en vez de agotador. Otro factor es el desbalance de la carga entre integrantes, que se corrige repartiendo las tarjetas por módulo y no por capa, para que ambos toquen servidor e interfaz.

B. PARTE II

4. Objetivos

En este apartado debes definir objetivos generales y específicos del Proyecto APT. Es importante aclarar que los objetivos se deben plantear en forma clara, concisa y sin dar mayores explicaciones, es decir, deben entenderse por sí solos. Se sugiere redactarlos utilizando un verbo en infinitivo, pues ello obliga a precisar acciones concretas.

Objetivo general	Desarrollar una intranet corporativa que centralice la comunicación interna, la gestión documental y el control de acceso por roles de una organización con equipo distribuido, garantizando que cada usuario accede únicamente a la información que le corresponde.
Objetivos específicos	1. *Modelar la base de datos relacional que sostiene la plataforma, con sus entidades, relaciones y claves foráneas.* 2. *Implementar la autenticación con tokens de acceso, el cierre de sesión y la validación de la sesión activa.* 3. *Implementar el control de acceso por rol, con políticas de autorización a nivel de modelo y de ruta.*

	4. *Implementar la mensajería interna en tiempo real, con adjuntos y contador de mensajes no leídos persistido en base de datos.* 5. *Implementar la gestión documental con carpetas jerárquicas por departamento y control de visibilidad por archivo.* 6. *Implementar la administración de usuarios, roles y departamentos, con validación del tipo real de los archivos subidos.* 7. *Validar el sistema mediante pruebas automatizadas de autorización, seguridad y reglas de negocio.* 8. *Documentar la arquitectura del sistema y su modelo de datos, y ordenar el proyecto con control de versiones, revisión de código y documentación continua.*

5. Metodología

En el siguiente apartado deberás describir la metodología, propia de tu disciplina, que utilizarás para resolver el proyecto APT antes descrito, incluyendo las etapas y métodos de trabajo.

Descripción de la Metodología

La metodología empleada es iterativa y orientada a resultados, con el trabajo organizado en iteraciones que cierran cuando cumplen un criterio de salida y no cuando se cumple una fecha. Cada iteración tiene un objetivo acotado y verificable, y el avance se mide por tarjetas completadas, no por semanas cumplidas. El plan se gestiona en un tablero Kanban, donde cada tarjeta declara sus criterios de aceptación antes de escribirse el código.

Esta decisión fue un ajuste del plan inicial. Partimos con una carta Gantt con semanas, pero no resultó operativa: estimar cuánto tomaría cada tarea era lo más costoso, y cualquier atraso se propagaba en cascada. El desarrollo técnico se apoya en Git, con ramas por funcionalidad y pull requests con revisión cruzada antes de integrar a la rama principal. Cada tarjeta del backlog corresponde a commits identificados con el mismo ID, de modo que el historial del repositorio funciona como registro de avance y de evidencia.

Fases de trabajo:

1. Definición y cimientos. Problema, alcance, arquitectura general, monorepo con las dos aplicaciones, migraciones iniciales y autenticación completa.
2. Seguridad y roles. Cierre de las brechas de autorización: rutas protegidas, políticas por modelo, control de roles, validación de archivos y límite de intentos de inicio de sesión.
3. Cobertura funcional. Módulos de comunicación, publicaciones, documentos y administración, con la interfaz construyéndose contra la API real.
4. Documentación y validación. Documento de arquitectura, modelo de datos, requisitos, historias de usuario, matriz de trazabilidad y pruebas con usuarios reales.
5. Cierre. Identidad visual, accesibilidad, informe final y entrega.

Funciones, tareas y responsabilidades de cada integrante:

- **Benjamín Herrera:**
  a. **Backend, base de datos, arquitectura y seguridad:** Configuración del servidor y del modelo de datos; autenticación y policies; API de tareas, mensajes, documentos y publicaciones; implementación de la seguridad; pruebas automatizadas del backend; documento de arquitectura y modelo de datos; despliegue y configuración de los servicios.

- **Julián Tapia:**
  a. **Frontend, UI/UX e integración en tiempo real:** Layout y componentes base; navegación adaptable al rol; recorrido de bienvenida; tablero de tareas y detalle de tarea; publicaciones con comentarios y adjuntos; vistas de documentos, departamentos, usuarios y roles; conexión WebSocket en el cliente; coherencia visual y accesibilidad.
- **Compartidas:**
  a. **Modelo de datos, integración, planificación y cierre:** Definición del modelo de datos y de los contratos de la API; integración entre las dos aplicaciones; planificación por iteraciones, control de versiones y revisión de código; evidencias, validación con usuarios e informe final

6. Evidencias

A continuación, describe qué evidencias serán evaluadas en el informe de avance y en el informe final de tu proyecto APT. Estas evidencias deben ser acordadas con tu docente. Se entenderá por evidencia los productos que se desarrollen durante el proyecto y cuyo propósito sea visibilizar o documentar cómo se ha implementado el trabajo.

Tipo de evidencia (avance o final)	Nombre de la evidencia	Descripción	Justificación
Avance	Repositorio con historial de trabajo por tarjeta	Repositorio monorepo con el backend, el frontend y la documentación, con el historial de commits donde cada tarjeta cerrada del backlog es identificable por su ID en el mensaje del commit, y los pull requests con su revisión cruzada.	Permite recorrer el proyecto tarjeta por tarjeta y verificar el orden real en que se resolvió cada problema, además de evidenciar que el versionado y la revisión entre integrantes se aplicaron y no solo se declararon.
Avance	Suite de pruebas automatizadas del backend	Archivos de prueba que verifican los caminos críticos: autorización por rol (respuesta 403 ante un usuario sin permiso), límite de intentos de inicio de sesión, rechazo de archivos cuyo tipo real no corresponde, persistencia de mensajes leídos y seeders de datos.	Da cuenta del cumplimiento del objetivo de validar el sistema, y sobre todo de la competencia de resolver vulnerabilidades sistémicas: la evidencia de que la protección de los endpoints no es solo una declaración en el código, sino un comportamiento verificado.
Final	Documento de arquitectura de software y modelo de datos	Documento que describe el sistema con el modelo de vistas 4+1, incluyendo las capas de la aplicación, las decisiones de diseño justificadas, el registro de riesgos con sus mitigaciones y los flujos críticos; más el diagrama entidad-relación y el esquema SQL consolidado, generado a partir de las migraciones reales.	Es la evidencia del resultado final y de su pertinencia: demuestra que el sistema resuelve el problema planteado y no solo que compila. Las pruebas con usuarios son la única forma de comprobar que la interfaz cumple su propósito.

7. Plan de Trabajo

En la siguiente tabla define la planificación de tu Proyecto APT de acuerdo a lo requerido.

Plan de Trabajo Proyecto APT

Competencia o unidades de competencias	Nombre de Actividades/Tareas	Descripción Actividades/Tareas	Recursos	Duración de la actividad	Responsable[1]	Observaciones
Construir modelos de datos	Definir el modelo de datos inicial	Crear las migraciones del esquema relacional: usuarios, roles, departamentos, carpetas, documentos, conversaciones, mensajes, lecturas, publicaciones, adjuntos y comentarios.	Laravel, Eloquent, MariaDB	1 semana	Benjamín Herrera	El esquema completo se definió antes de la interfaz, lo que evitó rehacer consultas. Facilitador: el modelo se pudo probar de inmediato con los seeders.
Desarrollar una solución de software	Autenticación con tokens	Implementar login con emisión de token, validación de sesión y cierre de sesión, con almacenamiento hasheado del token.	Laravel Sanctum, hashed tokens	1 semana	Benjamín Herrera	Decisión documentada: tokens de acceso personal en vez de Sanctum SPA, porque el cliente y el servidor corren en puertos distintos.
Construir el modelo arquitectónico	Sidebar base y recorrido de bienvenida	Construir la navegación principal y el tour de onboarding con el que el usuario nuevo descubre la herramienta.	Next.js, driver.js	1 semana	Julián Tapia	El tour resultó más difícil de lo previsto: un recorrido que no aporta molesta y se cierra sin usarse.
Resolver las vulnerabilidades sistémicas	Límite de intentos de inicio de sesión	Aplicar un rate limit de 5 intentos por minuto y combinación de IP y correo, con	Laravel throttle, test PHPUnit	1 semana	Benjamín Herrera	Proveniente de la revisión de seguridad. Sin esto el login quedaba expuesto a

[1] En caso de que el Proyecto APT sea grupal, en esta columna deben indicar el nombre de los responsables de cada tarea o actividad. Esto posteriormente permitirá diferenciar la evaluación por cada integrante.

		respuesta clara al superar el límite.				intentos automatizados.
Resolver las vulnerabilidades sistémicas	Autorización de usuarios, departamentos y roles	Restringir las operaciones de administración a administradores mediante middleware, dejando la lectura de departamentos y roles abierta a usuarios autenticados.	Middleware, rutas API, tests	2 semanas	Benjamín Herrera	El criterio original era más estricto; se relajó la lectura porque el listado de áreas lo necesita todo el equipo. Buscar todos los puntos sin autorización fue lo más costoso.
Resolver las vulnerabilidades sistémicas	Autorización de documentos y validación real de archivos	Implementar una policy que permita escribir solo al administrador o al dueño del archivo, y validar el tipo MIME real de las subidas rechazando con 415 cuando no corresponde a la extensión.	Policies, Form Requests, PHPUnit	2 semanas	Benjamín Herrera	La validación del MIME real era un punto ciego: antes se confiaba en la extensión declarada.
Resolver las vulnerabilidades sistémicas	Persistencia de mensajes leídos	Registrar en base de datos qué mensajes ha leído cada usuario y calcular desde ahí el contador de no leídos.	Modelo de lecturas, API, frontend	1 semana	Benjamín Herrera y Julian Tapia	Ajuste de diseño: el contador vivía en el navegador y se perdía al cerrar la pestaña, lo que lo volvía inútil en una herramienta de trabajo.
Resolver las vulnerabilidades sistémicas	Modelado de los roles canónicos	Definir los cuatro roles (Administrador, Líder, Colaborador, Nuevo Ingreso), su seeder y los indicadores de usuario asociados.	Seeder, migraciones, modelo	1 semana	Benjamín Herrera	La sustitución del chequeo de administrador por un método único se extendió a rutas, permisos y frontend, porque el valor estaba repetido en varios lugares.
Construir el modelo arquitectónico	Centralización de la lógica de permisos	Reunir la decisión de permisos en el modelo de usuario y cubrir con pruebas los cuatro roles.	Modelo, PHPUnit	1 semana	Benjamín Herrera	Eliminó la condición repetida en varios puntos del código, que era la principal fuente de inconsistencias.

Construir el modelo arquitectónico	Sidebar y rutas según el rol	Mostrar en la navegación solo las opciones que le corresponden a cada usuario, y proteger las rutas según el permiso.	Next.js, App Router	1 semana	Julian Tapia	Requiere volver a tocar el frontend cada vez que cambia un permiso, así que quedó atado a los cambios de la API.
Construir programas y rutinas	Integración de la rama de desarrollo	Integrar el trabajo mediante pull request con revisión cruzada y resolver los conflictos de formato y de lint.	Git, GitHub, pull requests	1 semana	Benjamín Herrera y Julian Tapi	El primer PR mergeado evitó subir código roto a la rama principal. El lint acumulado apareció al final y hubo que separarlo del cambio de roles.
Construir programas y rutinas	Rediseño de las siete vistas del dashboard	Rediseñar las vistas con un sistema de tokens visuales semánticos, coherencia tipográfica y correcciones de legibilidad reales.	Next.js, Tailwind CSS, tokens CSS	2 semanas	Julian Tapia	Fue el cambio más grande del repositorio y quedó en un solo commit por presión de tiempo, lo que dificultó la revisión.
Construir modelos de datos	Esquema consolidado y modelo entidad-relación	Consolidar el esquema SQL definitivo y elaborar el diagrama ER a partir de las migraciones.	Mermaid, SQL, migraciones	1 semana	Benjamín Herrera	Generarlo desde las migraciones y no dibujado a mano garantiza que el modelo documentado sea el que realmente existe.
Construir el modelo arquitectónico	Identidad de marca y coherencia de textos	Incorporar el isotipo, el favicon, los tokens de marca y la corrección de textos y tildes en todas las vistas.	Next.js, metadata, CSS	1 semana	Julian Tapia	El nombre del proyecto pasó de PR-Intra a Coherev en esta etapa, junto con la identidad visual.
Construir programas y rutinas	Paginación, color por departamento y modal de publicaciones	Agregar paginación a los listados, identificar visualmente cada departamento y resolver la creación de publicaciones en un modal.	Next.js, API	1 semana	Julian Tapia	Son mejoras de uso que no estaban en el alcance original y se priorizaron por el feedback interno.

Construir el modelo arquitectónico	Documento de arquitectura de software	Redactar el documento con el modelo de vistas 4+1: capas, casos de uso, vistas lógica, de desarrollo, de despliegue y de datos, atributos de calidad, flujos críticos, decisiones de diseño y riesgos.	Markdown, Mermaid, código	4 semanas (en paralelo)	Benjamín Herrera	Se resolvió una observación de la docente. Es el trabajo de documentación más pesado del proyecto y compite con el desarrollo por tiempo.
Construir programas y rutinas	Módulos funcionales pendientes	Implementar el módulo de tareas, el directorio de colaboradores y el panel de novedades para el usuario nuevo.	Laravel, Next.js	4 semanas	Pendiente de repartir	Son los tres módulos que la revisión identificó como faltantes. Se repartirán por módulo y no por capa, para que ambos toquen servidor e interfaz.
Construir el modelo arquitectónico	Documentación de planificación	Elaborar el documento de requisitos, las historias de usuario y la matriz de trazabilidad que conectan problema, requisito, historia, funcionalidad y prueba.	Markdown	3 semanas	Pendiente de repartir	Corresponden a observaciones directas de la docente y son las de mayor riesgo en la evaluación.
Construir programas y rutinas	Validación con usuarios reales	Realizar pruebas de usabilidad con personas reales, registrar métricas y corregir los defectos detectados.	Formularios, hoja de registro	3 semanas	Pendiente de repartir	No se puede resolver leyendo documentación: requiere que alguien use la aplicación. Depende de conseguir usuarios.
Construir el modelo arquitectónico	Accesibilidad y experiencia de usuario	Sidebar móvil, navegación por teclado, contraste verificado, foco visible y errores por campo en los formularios.	Next.js, axe-core	2 semanas	Pendiente de repartir	Depende de que la interfaz esté estable, por lo que va después del rebranding.
Gestionar proyectos informáticos	Cierre y entrega	Ensayo de la presentación, informe final, actualización de la	Documentación, repositorio	2 semanas	Benjamín Herrera y Julian Tapia	Depende del cierre de todas las

		documentación y entrega del repositorio.				iteraciones anteriores.